SDG 9.1 · RESILIENT INFRASTRUCTURE

# Hydra Stream Feature Guide

A region-aware, Kafka-style message broker that keeps data inside its legal jurisdiction, routes around slow links, and keeps the surviving regions running when a whole region fails. This guide explains every feature: what it does, how it works, how it compares with Apache Kafka and Apache Pulsar, and the measurements behind each claim.

**0**
acknowledged messages lost or duplicated across 6 chaos scenarios (~904k msgs)

**11×**
lower live-traffic p95 delay during recovery (57 ms vs 632 ms FIFO)

**8.8×**
fewer duplicate reprocessings after a consumer crash (67 vs 587, median)

**18/18**
simulator predictions matched the real cluster after a region failure

Version: Milestone 7 · Go, standard library only (no broker, queue or embedded database) · runs on one laptop
Repository: `hydra-stream` · design notes in `NOTES.md` · raw results in `results/`

# Contents


1. **How to read this guide** — the claims policy and baselines
2. **Architecture in one page**
3. **Comparison at a glance** — Hydra Stream vs Kafka vs Pulsar
4. **Part A — the core platform**
    1. Segmented append-only log (no database)
    2. In-region replication with lease + epoch fencing
    3. Independent regions and cross-region quorum for global topics
    4. Latency-aware relay routing
    5. Residency guard on every data path
    6. Signed segment manifests and `residency_audit`
    7. Fast, specific errors
    8. Idempotent producer
    9. Chaos harness and correctness checker
    10. Measured against a naive single-controller design
    11. One-command demo and dashboard
5. **Part B — the seven improvements**
    1. Granular failover checkpoints
    2. Adaptive replication bandwidth controller
    3. Message-level jurisdiction enforcement
    4. Predictive backlog and storage forecasts
    5. Pre-deployment failure and policy simulator
    6. Cross-region payload deduplication
    7. Per-message delivery and durability modes
6. **Part C — extras**: metrics, compliance evidence, one-command signed policies
7. **Measured results** — full tables
8. **Limitations and honest notes**
9. **How to reproduce every number**


## 1. How to read this guide

Every claim in this guide is backed by a test, a benchmark or a chaos run in the repository, and the command that reproduces it is listed in section 9. Two rules shaped the comparisons:

- **Fair baselines on the same code.** Running Kafka or Pulsar was not allowed in this project, so every "before vs after" number compares Hydra's algorithm with the standard alternative implemented as a mode of the *same* code: a FIFO replicator, a fixed replication rate limit, a single committed offset. That isolates the algorithmic difference and avoids comparing a laptop prototype's raw speed with a tuned production system.
- **Medians and tails, not best runs.** Benchmarks repeat and report the median, and p95/p99 where tails matter.

When this guide says Kafka or Pulsar "does not" do something, it refers to their standard, documented features (Kafka: committed offsets, static replication throttles, MirrorMaker 2 / cluster linking for geo; Pulsar: namespace-level geo-replication with allowed clusters, individual acks for shared subscriptions, dispatch-rate limits). Both are mature systems with ecosystems Hydra does not try to match; the comparison is about the specific problems in this brief: **jurisdiction, regional failure and recovery**.

## 2. Architecture in one page

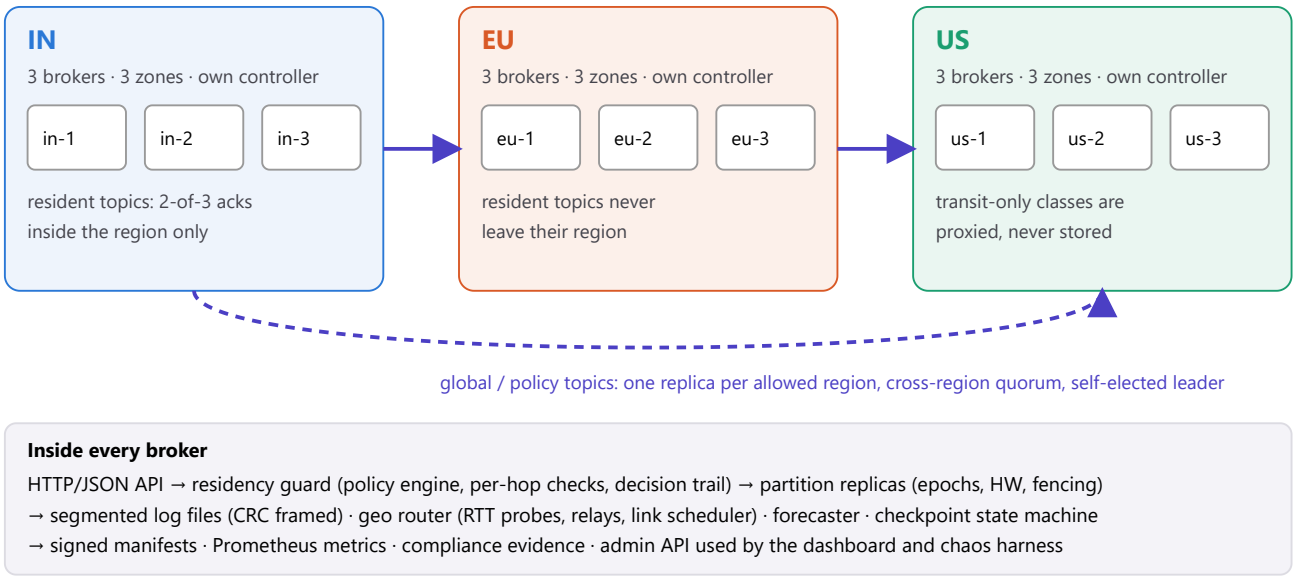



Figure 1. Three independent regions on one laptop (configurable). Each region elects its own controller with majority-granted leases; no global controller exists, so no single region's failure can freeze another.

**Two kinds of topic.** A *resident* topic ( `residency: IN` ) lives on three brokers in three zones of one region and commits on 2 of 3. A *cross-region* topic ( `global` or a `policy:<class>` spanning regions) has one replica per allowed region and commits on a majority of regions, with a leader elected among the replicas themselves (pre-vote + check-quorum), so it survives any single region's loss without a global coordinator.

**Everything is config-driven.** Regions, brokers, zones and addresses come from `configs/cluster.local.json` ; the same binaries run as local processes (the demo) or in containers.

**Constraints honoured.** No existing broker or queue, no embedded database: the log is plain segment files written by our own code; only the Go standard library is used (HTTP, JSON, Ed25519, SHA-256, CRC32-C).

### 3. Comparison at a glance

| Capability | Kafka (standard) | Pulsar (standard) | Hydra Stream |
|---|---|---|---|
| Data residency unit | whole cluster (mirror selected topics) | namespace/topic allowed-cluster list | **per topic and per message, storage ≠ transit** |
| Signed, versioned, fail-closed policy | no | no | **Ed25519, no rollback, expiry** |
| Per-hop jurisdiction audit record | no | no | **every deny + sampled allows, signed evidence** |
| Isolated region loses a broker | stretched cluster: waits for remote controller | per-cluster | **fails over inside the region (measured: 0.59 s vs frozen 8.25 s)** |
| Region failure isolation | per-cluster; stretch clusters share a controller quorum | per-cluster; shared config store for geo | **independent regions; global data by region quorum** |
| Replication bandwidth control | static throttle rate | static dispatch-rate limits | **adaptive (AIMD on live delay) + urgent overdraft** |
| Consumer progress after failover | single committed offset | individual acks (shared subs), cursor not owner-fenced | **ranges + in-flight + owner epoch fencing** |
| Recovery forecasts | metrics only | metrics only | **time-to-clear with range, time-to-full, advice** |
| What-if simulator | no | no | **same code as brokers, validated 18/18** |
| Cross-region dedup | no | no | **hash references, full payloads on disk** |
| Per-message durability | per producer (acks) | per topic/namespace | **per request, with policy floor** |
| Idempotent producer | **yes** | **yes (dedup)** | **yes (on par)** |
| Ecosystem, connectors, scale | **very large** | **large** | prototype |

# Part A — The core platform

These are the foundations that the seven improvements build on. Each is summarised in the same structure: what it does · how it works · why it matters / how it compares · evidence.

## A1. Segmented append-only log — no database

**What.** Every partition is a directory of segment files; records are appended and never rewritten.

**How.** Each record is framed with its length and a CRC32-C checksum and carries offset, leader epoch, timestamp, flags, producer id and sequence, headers, key and value. Segments roll at a configured size. On start-up the log scans the last segment and truncates any torn tail whose checksum fails, so a power cut never leaves a corrupt record visible. An in-memory index maps offsets to file positions; `EpochAt` answers "which leader wrote this offset", which replication uses to detect divergence.

**Why it matters.** The brief banned embedded databases; plain files also make the residency audit trivial — data is exactly where its files are.

**Evidence.** Hard kill during 400,000-message writes: every message and offset intact. Single broker: ~700k msg/s produce on loopback.

## A2. In-region replication with lease + epoch fencing

**What.** A region's controller is elected without Raft libraries; partition leaders fail over in well under a second and a stale leader that comes back can never acknowledge a write.

**How.** A candidate becomes controller only with **leases granted by a majority** of the region's brokers; promises are persisted so a restarted broker can't grant twice. Every metadata change, leader election, write and replication request carries an **epoch**; anything with an older epoch is rejected. Leaders push batches with the previous offset and epoch; followers truncate to the high watermark on divergence. Before electing a new leader, the controller fences the old one.

**Why it matters.** The controller is not a single point of failure (any majority can elect one), and the classic split-brain — an old leader paused by GC returning and accepting writes — is closed by epochs, not clocks.

**Evidence.** Tests for controller crash, leader crash and a stale leader after a pause, each run ×8: new leader in 175–606 ms, 0 acknowledged messages lost, stale leader never acks and is fenced on return.

> **CLOCK ASSUMPTION**
>
> Leases depend on clocks advancing at roughly the same rate (not on synchronised time). Safety does not depend on clocks: epochs fence writes even if a lease is wrongly believed valid.

## A3. Independent regions and cross-region quorum

**What.** Each region runs its own control plane. Global topics keep accepting writes when any one region dies or is cut off.

**How.** A cross-region partition has one replica per allowed region. Its replicas elect a leader among themselves with Raft-style votes, **pre-vote** (an isolated replica can't inflate epochs) and **check-quorum** (a leader that loses touch with a majority steps down). Writes commit when a majority of regions has them.

**Compared with Kafka/Pulsar.** A stretched Kafka cluster shares one controller quorum across regions; Pulsar geo-replication is asynchronous between clusters (a region's unreplicated writes are at risk when it fails). Hydra gives global topics synchronous majority durability *and* keeps every region's local data fully independent.

**Evidence.** Region crash ×5: global leader re-elected in 0.32–0.45 s, surviving resident topics stall ≤ 42 ms, 0 loss. Region isolation ×5: the isolated region keeps serving its own data (1–2 ms writes), and global writes there fail in ~1–2 ms with `REGION_UNAVAILABLE` instead of hanging.

### A4. Latency-aware relay routing

**What.** When a direct inter-region link gets slow, global traffic automatically goes through a third region.

**How.** Brokers probe every region continuously and keep an EWMA of round-trip time. A relay is chosen when it is at least 30% faster *and* saves at least 10 ms, with hysteresis so routes don't flap. The rule is one pure function, `geo.ChooseRelay`, shared with the simulator. Relays are only used through regions the data's policy allows to transit, at most 2 hops, with loop detection.

**Evidence.** IN↔EU slowed by 150 ms each way (301 ms RTT): rerouted via US within 135–180 ms; fetch p50 1.1 ms before vs 1.3 ms during the fault.

### A5. Residency guard on every data path

**What.** A single guard is consulted before data is stored, replicated, forwarded, proxied or relayed — including when configuration is wrong.

**How.** Placement only considers eligible brokers; the leader checks the follower's region before sending; the follower checks again before accepting; relays check transit and the destination's storage permission. A test deliberately puts an EU broker into India's peer list: India's data is still never placed on, replicated to, relayed through or accepted by it.

**Why it matters.** Defence in depth: a single misconfiguration can't leak data, because the receiving side refuses too.

### A6. Signed segment manifests and residency_audit

**What.** An offline auditor proves where every byte lives.

**How.** Each broker signs (Ed25519) a manifest of every segment it stores. `residency_audit` scans each data directory, verifies each live broker's manifest against pinned keys, and judges each topic against its residency or signed policy class. If it can't verify a policy it **fails** rather than passing silently. Exit code 1 on any violation.

**Evidence.** Clean cluster: PASS, 9/9 manifests verified. India's data copied onto an EU broker: FAIL with 6 violations. Audit every 5 s during all chaos faults: PASS before/during/after in all 6 scenarios.

### A7. Fast, specific errors

Clients never wait on a timeout to learn the obvious. `WRONG_REGION` (with the brokers to use), `REGION_UNAVAILABLE` with a reason ( `region_unreachable` , `region_isolated` ), `NOT_LEADER` with a hint, `STALE_EPOCH` , `NOT_ENOUGH_REPLICAS` , `RESIDENCY_VIOLATION` with a reason, `POLICY_UNAVAILABLE` . In chaos runs the slowest error answer was 42 ms; cross-region "region down" answers arrive in ~55 ms.

### A8. Idempotent producer

Producer id + per-partition sequence numbers; the leader remembers the last 8 batches per producer (rebuilt from the log after restart), so retries after a failover never duplicate. On par with Kafka's and Pulsar's idempotence; required for the "0 duplicates" result below.

### A9. Chaos harness and correctness checker

**What.** `hydra-chaos` starts all 9 brokers as real processes, runs six streams of load, injects a fault, heals it, then verifies every acknowledged message: present, exactly once, in order — and runs the residency audit before, during and after.

**Scenarios.** baseline, kill-broker (the leader), kill-zone (the controller's zone), kill-region, isolate-region, slow-link, and isolate+kill-leader (cut a region off, then kill its resident leader — see A10).

| Scenario | Acked | Lost | Dups | Order | Max stall surviving | Max stall global | Audit |
|---|---|---|---|---|---|---|---|
| baseline | 148,560 | 0 | 0 | 0 | 14 ms | 13 ms | PASS ×3 |
| kill-broker | 146,820 | 0 | 0 | 0 | 15 ms | 75 ms | PASS ×3 |
| kill-zone | 149,140 | 0 | 0 | 0 | 16 ms | 751 ms | PASS ×3 |
| kill-region | 149,660 | 0 | 0 | 0 | 16 ms | 195 ms | PASS ×3 |
| isolate-region | 153,580 | 0 | 0 | 0 | 15 ms | 156 ms | PASS ×3 |
| slow-link | 155,860 | 0 | 0 | 0 | 14 ms | 308 ms | PASS ×3 |

Run `results/chaos-20261031-183900.md`, with signed policies installed. In kill-region, India's own stream waits for India to return (8 s) by design — its data may not fail over abroad — while its clients get errors within 42 ms. Surviving regions never stalled more than 16 ms: well inside the 1 s target.

## A10. Measured against a naive single-controller design

**What.** The same binary run as one stretched cluster: `-coordination global` puts all 9 brokers in one coordination group, so a single controller decides every leader in every region (the common "one cluster across regions" setup). Same replication, same storage, same scenarios — only the control plane differs. `hydra-chaos -mode compare` runs both on fresh clusters.

| Scenario | Hydra: max stall surviving / global | Baseline: max stall surviving / global | Result |
|---|---|---|---|
| no fault | 33 / 39 ms | 21 / 14 ms | PASS / PASS |
| kill-broker | 13 / 778 ms | 19 / 75 ms | PASS / PASS |
| kill-zone | 13 / 17 ms | 23 / 28 ms | PASS / PASS |
| kill-region | 13 / 199 ms | 34 / 209 ms | PASS / PASS |
| isolate-region | 21 / 155 ms | 20 / 159 ms | PASS / PASS |
| slow-link | 26 / 311 ms | 16 / 311 ms | PASS / PASS |
| **isolate+kill-leader** | IN back in **590 ms** | IN frozen **8,250 ms** | PASS / FAIL |

**The deciding case — every run (3 of 3).** Cut India off, then kill the leader of India's own topic. India still has 2 of 3 brokers. Hydra's India controller elects a new leader inside India in ~0.6 s; with one global controller on the other side of the cut nobody can elect one, so India's data is frozen for the whole fault — exactly the "isolated region keeps serving its own data" requirement.

Final run `results/compare-20261101-084600.md`; both designs lost and duplicated nothing. Elsewhere the two designs are comparable on one laptop. The first compare run also showed a baseline failure in kill-broker (controller churn re-elected a live US leader, 8.3 s stall) that did not recur in three later runs, so it is not claimed. On one laptop regions are 0.5 ms apart; across a real WAN the stretched controller would also pay inter-region RTT on every lease renewal and election, which this setup can't show.

## A11. One-command demo and dashboard

`.\scripts\hydra.ps1 demo` builds everything, signs the policies, starts 9 brokers and load, and opens a dashboard with the live region map (RTTs, relays, cut links), throughput, leaders, audit status, events, and buttons to kill/restart/isolate regions and brokers, slow or cap links, run the audit — plus panels for every Part B feature (policy decisions, link scheduler, recovery forecast, what-if simulator).

# Part B — The seven improvements

## B1. Granular failover checkpoints

> **PROBLEM**
>
> Parallel consumers finish messages out of order. A single committed offset can only advance to the oldest unfinished message, so when a consumer crashes, every message finished above it is processed again — and a paused "zombie" consumer may later overwrite the new owner's progress.

**What it does.** Records exactly which messages are done, so a takeover redoes only work that was genuinely in flight, and guarantees only the current owner can move a checkpoint.

**How it works.**

- A checkpoint holds a **low watermark** (everything below is done), the **acknowledged ranges** above it (`RangeSet`: sorted, merged [from, to) ranges, capped at 4096), the **in-flight** offsets, and an **owner epoch**.
- `POST /v1/groups/{g}/claim` makes the caller owner and bumps the epoch. `commit` is a **compare-and-append** on the partition leader: it succeeds only if the epoch still matches, otherwise `STALE_EPOCH` (`stale_owner`). A checkpoint can't move backwards (`checkpoint_regression`).
- Checkpoints are records in an internal topic with the **same residency as the data** (`__cp.<residency>`), replicated like any partition and folded into current state by a replicated state machine on every replica. A failover of the checkpoint store itself loses nothing, and progress metadata never leaves the data's jurisdiction.

**Offsets 100–122 after a crash (green = finished, amber = in flight, grey = not started)**




Offset-only: committed offset = 103 → resume at 103 → the 13 finished messages above 103 are processed again.
Granular: low=103, acked=[104,108) [109,115) [116,119), in-flight={103,108,115} → only the 3 in-flight messages are redone.

**Why it is better.** Kafka's consumer groups store one committed offset, so out-of-order completion always turns into redelivery. Pulsar can track individual acknowledgements for shared subscriptions, but its cursor is not fenced by an owner epoch against a paused consumer, and it isn't scoped to the data's jurisdiction. Hydra combines exact progress, zombie fencing and residency in one mechanism.

**Evidence.**

| Experiment | Offset-only | Granular (Hydra) |
|---|---|---|
| Real cluster test (`TestGranularCheckpointsAndFencing`): messages redelivered after takeover | 100 | 1 |
| `hydra-bench`: 20k messages, 16 workers, heavy-tailed processing, 400 random crashes — duplicates, median | 587 | 67 |
| — p95 | 973 | 145 |
| Checkpoint size (median) | 1 offset | 9 ranges |
| Zombie consumer commits after takeover | accepted (overwrites) | `STALE_EPOCH` (refused) |

Limitation: no automatic partition assignment / rebalancing yet — a consumer claims a partition explicitly.

## B2. Adaptive replication bandwidth controller

> **PROBLEM**
>
> When a region comes back after an outage, its replicas pull a large backlog over the same inter-region link that live traffic needs. A plain replicator (FIFO) lets the catch-up flood delay live writes; a static rate limit is either too low (recovery takes forever) or too high (live traffic suffers), and is wrong as soon as the link changes.

**What it does.** Shares each inter-region link between traffic classes so live traffic stays fast, urgent control traffic is never delayed, and recovery uses everything that is left — automatically adjusting when capacity or load changes.

**How it works.**

- **Token bucket** per directed link enforces capacity; large sends may go into debt and are repaid (no fragmentation).
- **Start-time fair queuing** across classes: control (strict priority), urgent (weight 8), live (4), recovery (adaptive). Each class advances a virtual clock by bytes/weight; the lowest clock sends next. Work-conserving: idle share flows to others.
- **AIMD controller** every 100 ms on recovery's weight, driven by the smoothed p90 queueing delay of live traffic: multiply by 0.7 when live waits longer than the 20 ms target; add 0.25 when it waits under half the target and a backlog exists. Bounded to [0.25, 6] so recovery is never starved and never dominates.
- **Priority overdraft**: small control/urgent sends (≤ 4 KB) may overdraw the bucket from a separate budget (5% of capacity), so a 1 KB vote never waits behind a 128 KB catch-up chunk's debt. Found by our own benchmark (14% deadline misses before the fix).
- Batches are classified at the source: recovery when a follower is more than one batch behind, live otherwise; catch-up batches grow to 256 KB on cross-region links for efficiency.

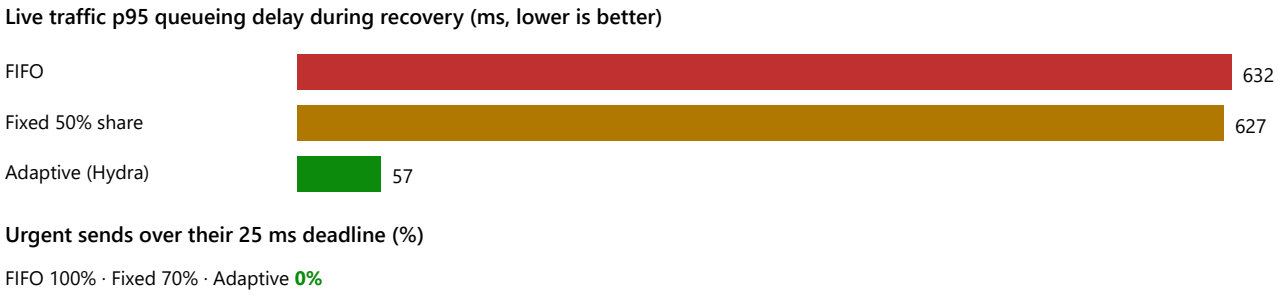



Figure 2. `hydra-bench bandwidth`: link 4 MB/s falling to 2.4 MB/s mid-run, 1.6 MB/s live traffic, 1 KB urgent every 50 ms, 10 MB recovery backlog with 4 senders; median of 3 runs.

| Mode | live p50 | live p95 | live p99 | urgent late | backlog cleared | utilization |
|---|---|---|---|---|---|---|
| FIFO (plain replicator) | 545 ms | 632 ms | 644 ms | 100% | 6.9 s | ≈100% |
| Fixed 50% recovery share (static throttle) | 460 ms | 627 ms | 640 ms | 70% | 7.9 s | 97% |
| **Adaptive (Hydra)** | **30 ms** | **57 ms** | **95 ms** | **0%** | 8.0 s | ≈100% |

**Why it is better.** Kafka's replication throttle and Pulsar's replicator dispatch-rate limits are static numbers an operator must pick in advance. Hydra's controller measures the effect on live traffic and adjusts every 100 ms, so it stays right when the link degrades (as in the benchmark, where capacity drops 40% mid-run).

**HONEST TRADE-OFF**

Adaptive clears the backlog about 1 s later than FIFO (8.0 s vs 6.9 s) because recovery yields to live traffic. That is the intended trade: live users see 11× lower p95 delay; catch-up finishes slightly later. Capacity is configured per link (dashboard "Cap links" or admin API); bandwidth is not estimated automatically.

### B3. Message-level jurisdiction enforcement

> **PROBLEM**
> Data-protection rules distinguish *where data is stored* from *where it may pass through*, differ per data class, change over time, and must be provable. Cluster-level replication lists can express none of that.

**What it does.** Each topic — and each message — carries a policy class that says where it may be stored, where it may transit, and the minimum durability. Every broker enforces it at every hop, records the decision, and refuses to move data when it can't evaluate the policy.

**How it works.**

- **Policies** (example classes in the demo): `public` (anywhere), `telemetry` (anywhere, acks=leader), `pii-in` (India only), `eu-personal` (stored in EU, may *transit* US, never stored there), `financial` (stored in IN and EU, never stored in or routed through US, acks=all).
- **Signed documents.** The administrator signs a versioned document with an expiry (Ed25519). Brokers verify the signature, refuse older versions (no rollback attack), reject unknown regions and duplicate classes. Missing, unsigned or expired ⇒ **fail closed**: `POLICY_UNAVAILABLE`, no data moves.
- **Producers can only narrow.** A per-message class header is intersected with the topic's policy: storage ∩ storage, transit ∩ transit, the stricter durability wins. A producer-requested relay must be a permitted transit region.
- **Every hop** runs the same guard: entry, forward, proxy, relay, replicate-send, replicate-accept, store. Relays are capped at 2 hops with loop detection. Regions that may only transit a class **proxy** requests to a storage region without writing to disk; regions that may not even transit it refuse at entry and redirect the client.
- **Placement** puts one replica in each permitted storage region (topped up to three), so `financial` survives losing either IN or EU while never touching US disks.
- **Decision trail**: every deny (and sampled allows) is appended to `decisions.log` with broker, topic, hop, peer region, policy and version, reason and evaluation time; `/admin/v1/compliance` returns a broker-signed evidence bundle; the residency audit judges `policy:<class>` data against the signed document.

| Situation (live demo) | What Hydra does |
|---|---|
| US client writes `payments` (financial) | Refused at entry: `WRONG_REGION` → IN/EU brokers; deny recorded (`transit_not_permitted`) |
| US client writes `eu-profiles` (eu-personal) | Proxied to EU in memory; never written to US disks |
| IN client writes `eu-profiles` | Refused at entry (IN may not even carry it); deny recorded |
| Producer asks to route financial data via US | `RESIDENCY_VIOLATION` (`route_not_permitted`) |
| Producer sends `acks=leader` on financial | Raised to `acks=all`; response states the mode applied |
| Policy document expires or is tampered with | Fail closed: `POLICY_UNAVAILABLE`; audit also fails closed |

**Why it is better.** Pulsar's geo-replication restricts a namespace or topic to a list of clusters; Kafka mirrors whole topics between clusters. Neither separates storage from transit, lets a single message narrow its rules, verifies a

signed policy, fails closed, or records each hop's decision. Hydra makes residency an enforced and auditable property of each message.

**Cost.** Per-hop policy check ( `hydra-bench policy` ): 0.1 µs (built-in residency), 0.3 µs (signed class), 0.8 µs (with per-message narrowing), 1.4 µs when an audit record is written — per batch, against 80–220 ms inter-region round trips.

**Evidence.** `TestJurisdictionPolicies` (fail closed without a document; placement avoids US; acks floor; producer route refused; transit-only proxying; narrowing); chaos suite and dashboard audit: PASS with policy topics live.

### B4. Predictive backlog and storage forecasts

> **PROBLEM**
>
> During an incident, operators need to know *when* things will be back to normal and whether something else (disk) will run out first — not just a lag number.

**What it does.** Every broker estimates, for each lagging replica and consumer group, when its backlog will clear (with a low/high range), when its storage will fill, and turns these into plain recommendations.

**How it works.** Every 500 ms the forecaster samples follower lag, consumer-group lag and data-directory size. Arrival and drain rates are tracked as exponentially weighted moving averages ($\alpha = 0.2$) with their variance; time-to-clear = backlog / (drain − arrival), with a range from ±1 standard deviation of the rates; time-to-full uses growth against the configured disk budget. Rules then produce advice, e.g. "*storage fills in ~4m, before the replication backlog clears (~9m): prioritise recovery bandwidth and throttle non-urgent producers now*", or "*follower is falling behind (in > drained)*". Shown at `/admin/v1/forecast` and on the dashboard; the same math powers the simulator.

**Why it is better.** Kafka and Pulsar export lag metrics; turning them into an ETA is left to external tooling. Hydra gives an explainable estimate and a recommended action where the data is, using math simple enough to verify by hand.

Limitation: assumes rates stay near recent averages; a step change takes a few seconds to be reflected.

### B5. Pre-deployment failure and policy simulator

> **PROBLEM**
>
> "What happens if we lose a region / change this policy / traffic grows 5×?" is usually answered by a production incident. A simulator that re-implements the system is only as good as its second copy of the rules.

**What it does.** Predicts, before anything changes: which partitions stay writable and from which client regions, which paths traffic will take and which data classes may not use them, where backlog builds and how long catch-up takes, how loaded each link is, and which stored data a proposed policy change would leave in a forbidden place.

**How it works.** It calls the **brokers' own code**: `placement` (where replicas go), `placement.Quorum` and the policy's `min_acks` (can it commit?), policy `Resolve` (who may store/transit), `geo.ChooseRelay` (path choice, extracted from the router so both use one function) and `forecast.TimeToClear`. Proposed policy changes are evaluated on an unsigned copy ( `policy.ForSimulation` ) that brokers never accept. Available as `hydra-sim` (offline, or `-broker` to read live topics and measured RTTs) and as the dashboard's "What-if" panel.

**Scenarios** (the guide's six, all supported): region offline · link 10× slower · region removed from allowed transit · 5× traffic · destination storage full · two regions recover; plus isolation, link cut and storage removal.

| What-if | Prediction (demo topology) |
|---|---|
| IN goes offline | 11/16 writable; IN-resident and pii-in stop (by law); both financial partitions stop because their policy demands acks=all; IN replicas catch up in ~3 s |
| IN↔EU 10× slower | IN↔EU relayed via US (300 vs 1200 ms) — but *not* for financial, which keeps the slow direct path |
| Remove US from eu-personal transit | "clients in US would get RESIDENCY_VIOLATION" — fix before signing |
| Remove EU from financial storage | Lists every EU replica that would hold now-forbidden data |
| 5× traffic + US down, 20 Mbit/s links | EU↔IN replication over capacity (160%): followers will fall behind; US needs ~2 min to catch up |
| EU and US recover after 5 min | 7 replicas each behind, 180 MB, ~32 s catch-up while sharing source links |

**Validated against reality.** `TestSimulatorMatchesCluster` builds a real 9-broker cluster with resident, global and policy topics, compares predicted with actual placement (identical for every partition), kills India, and checks every (partition, client region) pair: **18/18 predictions matched**. The first run disagreed on 2 — the simulator had ignored `financial`'s acks=all floor; the test exposed it and the simulator was fixed. That is the point of validating a model against the system.

**Why it is better.** Neither Kafka nor Pulsar ships a what-if tool for availability, routing and data-residency effects.

Limitation: backlog math is steady-state (average message size, no TCP dynamics).

### B6. Cross-region payload deduplication (stretch goal — implemented)

> **PROBLEM**
> Many workloads resend identical large payloads (documents, images, configuration blobs). Sending the same bytes across expensive inter-region links again and again wastes the scarcest resource during recovery.

**What it does.** On cross-region replication, a large value the follower already has is sent as a 32-byte hash reference.

**How it works.** For values ≥ 1 KB the leader remembers which hashes each follower is known to hold (bounded, 20k per follower, FIFO). Known values are sent as their SHA-256 ( `Ref` ) instead of the bytes. The follower looks the hash up in an index of its own log, reads that record back, **verifies the hash**, and appends the full value. If it can't resolve a reference (evicted, truncated, never seen) it answers `NeedFull` and the leader resends that batch in full — correctness never depends on the cache.

**Design choices.** Logs always hold complete payloads (no content-addressed blob store), so crash recovery, truncation, audit and retention never depend on dedup. References are scoped to one partition, so one tenant can never reference — or probe for — another's data. `-no-dedup` turns it off.

**Evidence.** `TestCrossRegionDedup` : 78 records sent as references, **655,200 bytes not sent** across regions, logs identical on both sides. Benefit is workload-dependent: zero for unique payloads (cost: one SHA-256 per large value).

**Why it is better.** Kafka and Pulsar compress batches but do not avoid re-sending payloads the remote side already stores.

### B7. Per-message delivery and durability modes

**What it does.** Each produce request chooses its durability and its traffic class, within the floor its policy sets.

**How it works.**

- `acks=leader` — acknowledged once the leader has it (lowest latency; the last batch can be lost if the leader dies).
- `acks=quorum` (default) — a majority of replicas (in-region: 2 of 3 zones; cross-region: a majority of regions).
- `acks=all` — every replica; writes stop (with explicit errors) while any replica is unreachable.
- A policy's `min_acks` is a floor: `financial` is always `all` , `telemetry` may use `leader` . The response reports the mode actually applied.
- Class hints ( `urgent` , `live` , `recovery` ) feed the link scheduler (B2).

**Why it is better.** Kafka sets acks per producer; Pulsar's durability is per topic/namespace. Hydra lets one producer send telemetry fast and payments safely on the same connection, while the compliance team's floor can't be bypassed.

**Evidence.** `TestAcksModes` ; the simulator and the real cluster agree that `acks=all` data stops accepting writes when a replica's region fails — a trade-off made explicit rather than hidden.

## Part C — Extras

- **Prometheus metrics** at `/metrics`: produced/proxied requests, policy denials, checkpoint claims/commits/fenced, per-link scheduler stats, replication lag.
- **Compliance evidence**: `/admin/v1/compliance` returns what a broker stores, under which policy version, and its jurisdiction decisions, signed with the broker's key.
- **One-command signed policies**: `hydra-policy ensure` creates the admin key and signs only when needed; brokers start with `-policies` (signature still verified); `push` installs a new version cluster-wide.
- **Containers**: `docker compose up --build -d` runs the same 9 brokers in containers (local testing only); the signing key never enters the image. Testing there exposed a missing connect timeout (a killed container drops packets), now fixed: leader-crash stall 4.05 s → 1.12 s.
- **Benchmarks with fair baselines**: `hydra-bench` (Part B numbers) and the chaos harness (Part A numbers).

## 7. Measured results — summary

| Area | Measurement | Result |
|---|---|---|
| Correctness under faults | 6 chaos scenarios, real processes, ~904k acked messages | 0 lost · 0 duplicates · 0 order violations |
| Freezing | max stall in surviving regions, any scenario | ≤ 16 ms (target < 1 s) |
| Explicit errors | slowest error answer during faults | 42 ms (target < 500 ms) |
| Global failover | region crash, re-election (tests ×5) | 0.32–0.45 s |
| Residency | audit before/during/after every fault | PASS in all 18 checks |
| Bandwidth (B2) | live p95 during recovery | 57 ms vs 632 ms FIFO |
| Bandwidth (B2) | urgent deadline misses | 0% vs 100% FIFO / 70% fixed |
| Checkpoints (B1) | duplicates after crash, median / p95 | 67 / 145 vs 587 / 973 |
| Jurisdiction (B3) | policy check cost per batch | 0.1–1.4 µs |
| Simulator (B5) | predictions vs real cluster | placement identical; 18/18 outcomes |
| Dedup (B6) | repeated-payload test | 655 KB not sent cross-region |
| vs single global controller | 7 scenarios, both designs | Hydra 7/7 PASS, baseline 6/7; cut-off region losing its leader: 0.59 s failover vs frozen 8.25 s (3 of 3 runs) |
| Throughput | single broker / 3-broker region / global | ~700k / ~185k / ~84k msg/s (laptop, loopback) |

## 8. Limitations and honest notes

- A prototype on one laptop over loopback; absolute throughput and latency won't transfer to WAN deployments. Inter-region delay is injected.
- Baselines are modes of Hydra's code, not Kafka/Pulsar binaries (not allowed in this project).
- Protocol is JSON over HTTP (simple, not the fastest); offsets index is in memory; no retention, compaction or authentication yet.
- Leases assume bounded clock drift; safety relies on epochs. Policy expiry uses wall-clock time.
- A global partition has one replica per region; losing a region and a broker in another region at once makes it unavailable until one returns.
- A policy change doesn't migrate existing data: the simulator lists what would be stranded and the audit flags it; moving it is an operator action.
- Consumer groups have no automatic rebalancing; link capacity is configured, not estimated.
- The demo keeps the admin signing key in `configs/`; production would keep it offline or in an HSM.

## 9. How to reproduce every number

```
go test ./...                                  # all unit + cluster tests (incl. simulator vs
cluster)
go test ./internal/broker -run "Jurisdiction|Checkpoints|AcksModes|Dedup|Simulator" -v
.\scripts\hydra.ps1 chaos                      # 7 scenarios, real processes → results\chaos-*.md
.\scripts\hydra.ps1 compare                    # same scenarios vs a single global controller
.\bin\hydra-bench.exe -out results\bench.md    # bandwidth, checkpoints, policy cost
.\bin\hydra-sim.exe -preset guide              # the six what-ifs
.\scripts\hydra.ps1 demo                       # dashboard at http://127.0.0.1:8080
.\bin\residency_audit.exe -online              # PASS / FAIL with exit code
```

Race detector: `go test -race ./...` (with gcc on PATH). Design rationale for every decision: `NOTES.md`.